Informatique Théorique

CM5: Représentation des réels à virgule fixe

Florian Bridoux

Polytech Nice Sophia

2025-2026

Table des matières

1 Nombre réels en différentes base

2 Représentation

Table des matières

1 Nombre réels en différentes base

2 Représentation

Les bases

Quand on écrit 834,26 en base 10, on représenté le réels :

$$8 * 100 + 3 * 10 + 4 * 1 + 2 * 0.1 + 6 * 0.01$$

Autrement dit,

$$8 * 10^2 + 3 * 10^1 + 4 * 10^0 + 2 * 10^{-1} + 6 * 10^{-2}.$$

D'une manière générale dans une base b, quand on écrit

$$d_m d_{m-1} d_{m-2} \dots d_1 d_0, d_{-1} d_{-2} \dots d_{-k}$$

où d_i est un symbole représentant un entier entre 0 et $b-1$, la valeur représentée est :

$$\sum_{-k \leq i \leq m} d_i * b^i.$$

Conversion d'une base vers la base 10

Quand on a une tel représentation, pour passer d'une base b à une base 10, il "suffit" de calculer :

$$\sum_{-k \leq i \leq m} d_i * b^i.$$

Exemples :

- $(101,11)_2 = (2^2 + 2^0 + 2^{-1} + 2^{-2})_{10} = (2^2 + 2^0 + \frac{1}{2} + \frac{1}{4})_{10} = (4 + 1 + 0.5 + 0.25)_{10} = (5.75)_{10}$.
- $(A.68)_{16} = (10 * 16^0 + 6 * 16^{-1} + 8 * 16^{-2})_{10} = (10 + 6/16 + 8/256)_{10} = (10 + 0.375 + 0.03125)_{10} = (10.40625)_{10}$.

Conversion d'une base vers la base 10

Alternativement, on peut calculer l'entier obtenu en ignorant la virgule et multiplier par b^{-k} avec k le nombre de chiffres après la virgule.

Exemples :

- $(101,11)_2 = ((2^4 + 2^2 + 2^1 + 2^0) * 2^{-2})_{10} = (\frac{16+4+2+1}{4})_{10} = (\frac{23}{4})_{10} = (5.75)_{10}$.
- $(A.68)_{16} = ((10 * 16^2 + 6 * 16 + 8) * 16^{-2})_{10} = (\frac{2560+96+8}{256})_{10} = (\frac{2664}{256})_{10} = (10.40625)_{10}$.

Représentation finie

Problème :

Certain réels n'ont pas de représentation avec une partie fractionnaire finie (dans aucune base).

Exemple : $\pi, e, \sqrt{2}, \ldots$.

Et certain nombre peuvent avoir une représentation avec une partie fractionnaire finie mais pas dans une autre...

Exemple :

$$(0.1)_3 = (1/3)_{10} = (0.33333333...)_{10}$$

La bonne nouvelle c'est que la conversion de la base 2 vers la base 10 ne pose pas de problème car 2 divise 10. Par contre, de la base 10 à la base 2...

Conversion de la base 10 vers une autre base

Comment convertir un réels n en base 10 en base b ?

On peut commencer par convertir la partie entière comme on l'a vu au cours précédent.

Ensuite on peut multiplier la partie fractionnaire par b de manière successive en retirant la partie entière. Les parties entières successives représentent les chiffres après la virgules (de gauche à droite).

Conversion de la base 10 vers une autre base

Exemple : $(12.6875)_{10} = (?)_2$

D'abord :

$$(12)_{10} = (1100)_2$$

Ensuite :

Multiplication	**partie entière**	**partie fractionnaire**
0.6875 * 2	1	0.375
0.375 * 2	0	0.75
0.75 * 2	1	0.5
0.5	1	0

Résultat : $(12.6875)_{10} = (1100, 1010)_2$

Conversion de la base 10 vers une autre base

Bien sûr, si le nombre n'a pas une représentation fractionnaire fini en base 10, ça boucle à l'infini...

$$(0.2)_{10} = (?)_2$$

Multiplication	**partie entière**	**partie fractionnaire**
0.2 * 2	0	0.4
0.4 * 2	0	0.8
0.8 * 2	1	0.6
0.6 * 2	1	0.2
0.2 * 2	0	0.4
0.4 * 2	0	0.8
0.8 * 2	1	0.6
0.6 * 2	1	0.2

Résultat : $(0.2)_{10} = (0.0011\ 0011\ 0011\ldots)_2$

Table des matières

1 Nombre réels en différentes base

2 Représentation

Virgule fixe

La représentation en virgule fixe utilise :

- 1 bit "de signe" ;
- m bits pour la partie entière ;
- n bits pour la partie fractionnaire ;
- **Le tout en complément à deux.**

Exemple de représentation en virgule fixe

Par exemple, en prenant 8 bits et $n = 4$:

- 0111 0000 représente 7.
- 0111 0110 représente
  $7 + 1/4 + 1/8 = 7 + 0.25 + 0.125 = 7.375$.
- 1111 0110 représente
  $-(0000\ 1010)_2 = -(1/2 + 1/8) = -(0.5 + 0.125) = -0.625$

Addition en virgule fixe

On veut s'assurer que l'addition "usuel" fonctionne normalement avec les réels en virgule fixe.
Prenons une représentation en virgule fixe sur 8 bits avec $n = 4$ (4 bits pour la partie fractionnaire) et en complément à deux.
Que donne l'addition de 1.375 et -0.3125 ?

$\rightarrow$ À vous de jouer.

Addition en virgule fixe

On veut s'assurer que l'addition "usuel" fonctionne normalement avec les réels en virgule fixe.
Prenons une représentation en virgule fixe sur 8 bits avec $n = 4$ (4 bits pour la partie fractionnaire) et en complément à deux.
Que donne l'addition de 1.375 et -0.3125 ?

- **Représentation en binaire :**
  - $1.375 \rightarrow 0001\ 0110$ $(1 + 1/4 + 1/8)$.
  - $-0.3125 \rightarrow 1111\ 1011$ $(-1/4 - 1/16)$.
- **Addition binaire :**

$$\begin{array}{r} 00010110 \\ +\ 11111011 \\ \hline (1)00010001 \end{array}$$

- **Résultat en binaire :** $0001\ 0001 \rightarrow 1 + 1/16 = 1.0625$.

Pourquoi l'addition fonctionne en virgule fixe ?

- Si x un nombre réel (partie fractionnaire fini), alors sa représentation en virgule fixe correspond également au code de l'entier $x \cdot 2^n$, où n est le nombre de bits de la partie fractionnaire.
- Quand on additionne deux nombres x et y en virgule fixe, l'opération binaire réalisée est :

$$\frac{(x \cdot 2^n) + (y \cdot 2^n)}{2^n} = x + y.$$

Exemple :

- $x = 1.375 \rightarrow 0001\ 0110$ $(x \cdot 2^4 = 22)$.
- $y = -0.3125 \rightarrow 1111\ 1011$ $(y \cdot 2^4 = -5)$.
- Résultat de l'addition binaire :

$$x + y = \frac{22 - 5}{2^4} = \frac{17}{16} = 1.0625.$$

Problèmes de la représentation en virgule fixe

1. **Problème d'arrondi :**
   - Communs à toutes les représentations (sauf les représentations sous forme de fraction mais elles sont gourmandes en ressources.)
   - Les changement de base peuvent casser des égalités (voir exemple python3).
   - Les problèmes d'arrondi peuvent s'accumuler...

Problèmes de la représentation en virgule fixe

2. Limitation des plages de valeurs :

- La représentation en virgule fixe impose une plage limitée de valeurs absolues (grandes ou petites).
- Exemples : - Si l'on réserve $m = 32$ bits pour la partie entière et $n = 32$ bits pour la partie fractionnaire, alors :

  Valeur maximale $= 2^{32} - 2^{-32}$, Valeur minimale (positive) $= 2^{-32}$.

  - Pour représenter la plage complète des `float` simple précision en Java (10^{38} à 10^{-45}), il faudrait environ 380 bits $(m + n)$ contre 32.

Conclusion

- La représentation en virgule fixe est rarement utilisée, car elle est inefficace pour les nombres avec des valeurs très grandes ou très petites.
- Pour remédier à ce problème, les représentations en virgule flottante (que l'on verra au CM suivant) sont préférées dans presque tous les cas.